

# Live Demo Runbook

A concise presenter guide for demonstrating the service in Postman and explaining the implementation choices.

10-15 minute demo

Postman steps

Presenter talking points

Expected outcomes

**Demo story:** authenticate, append evidence, query it, prove chain integrity, redact private values safely, export a verifiable bundle, and explain direct datastore tamper detection.

## Before the demo

1. Run `mvnw.cmd spring-boot:run`.
2. Import `postman/audit-log-service.postman_collection.json`.
3. Use base URL `http://localhost:8080`.
4. Keep Swagger UI open at `/swagger-ui.html` as a backup reference.

---

Local credentials: `admin` / `change-me-local`. Use only in local development.

## Demo map

1. POST /api/v1/auth/token -> receive JWT
2. GET /audit/verify -> valid chain
3. POST /audit -> append event + hash link
4. GET /audit?actorId=... -> query + pagination
5. PATCH /audit/records/{id}/... -> redact selected payload values
6. GET /audit/verify -> still valid
7. GET /audit/export?actorId=... -> signed verifiable bundle
8. Explain direct H2 tamper test -> invalid chain detected

## Step 1 - Authenticate

**1 Postman request:** POST http://localhost:8080/api/v1/auth/token

```
{
  "username": "admin",
  "password": "change-me-local"
}
```

**Expected:** 200 response containing `accessToken`, `tokenType: "Bearer"`, and `expiresIn`.

**Say:** "Audit actions are protected by a short-lived JWT. The token endpoint is public; all audit, retention, redaction, and export endpoints require `Authorization: Bearer <token>`."

**Postman check:** set Authorization type to *Bearer Token* and paste only the JWT itself. Do not paste the word `Bearer` into the Token field.

## Step 2 - Verify the chain before writing

**2 Postman request:** GET http://localhost:8080/audit/verify

**Expected:** 200 with `valid: true`. An empty chain is valid; an existing local database may show a positive `checkedRecords` count.

**Say:** "Verification always walks the complete ordered chain. It checks the predecessor link, event hash, and payload commitment for every stored record."

## Step 3 - Append an audit event

**3 Postman request:** POST http://localhost:8080/audit

```
{
  "eventType": "RECORD_UPDATED",
  "actorId": "demo-user",
  "resourceType": "customer",
  "resourceId": "customer-demo-1",
}
```

```
"payload": {  
  "source": "postman-demo",  
  "changedFields": ["email", "phone"],  
  "accountNumber": "1234567890",  
  "person": { "name": "Jane Doe", "ssn": "111-22-3333" }  
}
```

**Expected:** 201. Copy the returned `id` into a Postman variable named `recordId`.

**Say:** *"The server assigns the timestamp in UTC. It creates random salts and commitments for every scalar payload field, calculates the payload commitment root, links this event to the previous hash, and persists the final hash."*

## Step 4 - Query the event

**4 Postman request:** `GET http://localhost:8080/audit?actorId=demo-user&resourceType=customer&resourceId=customer-demo-1&page=0&size=10`

**Expected:** 200 page response. Inspect `content`, `page`, `size`, `totalElements`, and `totalPages`.

**Say:** "Filters combine with AND. Supported filters are actor, resource type, resource ID, event type, and inclusive UTC time range. Pagination is zero-based and capped at 100 records per page."

## Step 5 - Redact private payload fields

**5 Postman request:** `PATCH http://localhost:8080/audit/records/{{recordId}}/redactions`

```
{
  "fields": ["/accountNumber", "/person/ssn"]
}
```

**Expected:** 200. The response shows `[REDACTED]` at both paths, the paths in `redactedFields`, and no corresponding salts.

```
Why verification still works:
insert: commitment = SHA-256(random salt + canonical field value)
      payload root = hash of all path -> commitment pairs
      event hash includes payload root

redact: visible field becomes [REDACTED]
      original commitment remains
      matching salt is destroyed
```

**Say:** "The original secret is not retained as an accessible value. We preserve the hash-bound commitment but destroy the salt, so the service proves this field was committed originally without retaining enough material to re-test the secret value."

## Step 6 - Verify after redaction

**6 Postman request:** `GET http://localhost:8080/audit/verify`

**Expected:** `valid: true`.

**Say:** "This is the important privacy result: redaction is legitimate, yet normal modifications remain detectable. For an unredacted field we recompute its salted commitment. For a redacted field we require the marker, original commitment, and missing salt."

## Step 7 - Export a verifiable bundle

**7 Postman request:** `GET http://localhost:8080/audit/export?actorId=demo-user`

**Expected:** 200 with `records`, `chainProof`, `chainHeadHash`, `bundleDigest`, `signature`, `signingPublicKey`, and `signingKeyId`.

**Say:** *"The export includes the full chain proof, not just selected records. A recipient can apply the filter to the proof and confirm that no matching record was omitted. The canonical bundle digest is signed with Ed25519."*

**Important limitation:** the signing key is process-local in this prototype. For production, use a durable KMS/HSM-managed key and distribute the trusted public-key fingerprint separately.

## Step 8 - Retention behavior

**8 Postman request:** `POST http://localhost:8080/audit/retention/archive?olderThanDays=90`

**Expected:** 200 with a cutoff and archived record count. Fresh demo records normally produce `archivedRecords: 0`; that is expected.

**Say:** "Archive is a soft lifecycle state. Archived records disappear from ordinary queries, but they remain in storage, verification, and matching exports. Archive metadata is not part of the original event hash, so legitimate archival does not create a false integrity break."

## Step 9 - Direct datastore tampering proof

This step is best demonstrated by explaining the automated test rather than changing your demo database live.

Test sequence:

1. POST /audit creates a record through the HTTP API.
2. JdbcTemplate updates audit\_log\_entries.payload\_json directly in H2.
3. GET /audit/verify returns valid: false.
4. violationType is PAYLOAD\_COMMITMENT\_MISMATCH.

**Say:** "The database is not trusted. If an attacker changes raw stored JSON without reproducing its original salted field commitments, verification identifies the first inconsistent record. The test restores its temporary change so the rest of the integration suite remains isolated."

## Likely questions and crisp answers

| Question                                          | Answer                                                                                                                                                              |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Why server timestamps?                            | To prevent callers from backdating accepted audit events. Source-system time can be added in payload when required.                                                 |
| Why not delete archived entries?                  | Deletion breaks the evidence trail and makes completeness difficult to prove. Soft archive preserves auditability.                                                  |
| Why not simply hash the payload JSON?             | That would make a later privacy redaction invalid. Field commitments let the original payload be cryptographically bound while selected visible values are removed. |
| Can you prove the original redacted value?        | No, intentionally. Destroying the salt prevents later verification/guessing of that value; we prove only that a commitment existed in the original record.          |
| What happens with multiple application instances? | The current in-process synchronization protects one JVM. Production needs database serialization, optimistic retry, or a chain sequencer.                           |
| Is JWT authorization complete?                    | Authentication is implemented. Fine-grained roles, revocation, and identity-provider integration are documented future work.                                        |

## Close the demo

**Closing statement:** "The prototype demonstrates append-only audit evidence, queryability, tamper detection, policy-driven archival, privacy-preserving redaction, and independently checkable exports. The remaining production controls - multi-instance sequencing, KMS-backed keys, fine-grained roles, and legal-hold policy - are explicitly documented rather than hidden."

---

Useful backup URLs: `/swagger-ui.html`, `/api-docs`, `/actuator/health`, `/actuator/health/liveness`, and `/actuator/health/readiness`.